

Compiling targeted test module

```
/usr/bin/checkmodule: loading policy configuration from tmp/test.tmp
test.te:5:ERROR: 'unknown type unconfined_t' at token ':' on line 3199:
```

```
allow unconfined_t ify_t : file { relabello };
/usr/bin/checkmodule: error(s) encountered while parsing
configuration
make: *** [tmp/test.mod] Error 1
```

Oops! We have received an error while compiling our module!

This is because we have not defined the types and classes used elsewhere in our source file (*test.te*). But are these types not already defined in the policy? Doesn't *unconfined_t* exist already?

It does, but we need to explicitly define which types and classes are required for our module. To solve this error, let us define the types in the required section of our source file. We explicitly add the following lines to our source:

```
require {
    type unconfined_t;
    type fs_t;
}
```

So that the new *test.te* is:

```
[vbg@vbg test-selinux]$ cat test.te
policy_module(test, 1.1)
```

```
require {
    type unconfined_t;
    type fs_t;
}
```

```
type ify_t;
```

```
allow unconfined_t ify_t : file { relabello };
allow ify_t : fs_t : filesystem { associate };
```

Let us now compile this again:

```
[vbg@vbg test-selinux]$ make test.pp
Compiling targeted test module
/usr/bin/checkmodule: loading policy configuration from tmp/test.tmp
/usr/bin/checkmodule: policy configuration loaded
/usr/bin/checkmodule: writing binary representation (version 10) to
tmp/test.mod
Creating targeted test.pp policy package
rm tmp/test.mod.fc tmp/test.mod
```

This has successfully created our upgraded policy package. Thus the important points to remember while creating the code for a policy module are:

1. Specify all existing types of the policy in a required

section in the source file

2. Specify all new types to be created outside the required section

Let us now upgrade the version in the policy and check:

```
[root@vbg test-selinux]# semodule -u test.pp
[root@vbg test-selinux]# semodule -i | grep test
test 1.1
```

That shows that our latest version is now loaded. Let us now try to re-label */var/www/html/index.html* from *httpd_sys_content_t* to *lfile_t*:

```
[root@vbg test-selinux]# semanage l
[root@vbg test-selinux]# chcon -t lfile_t /var/www/html/index.html
chcon: cannot access '/var/www/html/index.html': Permission
denied
```

Again, there are some more errors! Is it that SELinux is too cumbersome to troubleshoot? Till when do we need to keep on looking at these errors and generate rules? This would be a very demotivating and time-consuming task. Fortunately, a splendid tool called *audit2allow* comes to our rescue.

audit2allow generates SELinux policy allow rules from the logs of denied operations. It parses the log file and creates corresponding allow rules. Let us repeat the above steps to generate AVC Denial logs in the audit log file.

```
[root@vbg test-selinux]# setenforce 0
[root@vbg test-selinux]# > /var/log/audit/audit.log
[root@vbg test-selinux]# cat /var/log/audit/audit.log | audit2allow

#===== unconfined_t =====
allow unconfined_t ify_t:file getattr;
```

audit2allow has many helpful options. Do ensure that you go through the man page in detail. To generate the "required" section for already existing system types and classes, use the option *-r* with *audit2allow*:

```
[root@vbg test-selinux]# cat /var/log/audit/audit.log | audit2allow -r

require {
    type unconfined_t;
    type ify_t;
    class file getattr;
}

#===== unconfined_t =====
allow unconfined_t ify_t:file getattr;
```

We can redirect these required statements to our policy module source file: